Async, Promise, and Future - Asynchronous Task Management

Theory, Edge Cases & Code Examples

- ✓ Overview
- ✓ Launch Policies
- ✓ Promise-Future Communication Channel
- ✓ std::future vs std::shared_future
- ✓ Thread Pool Building Block

→ *Swipe to learn* →

Async, Promise, and Future - Asynchronous Task Management

Theory

📌 1. The Async Trio - Overview

Three Related Components (C++11):

Component	Role	Purpose
std::future	Receiver/Consumer	Handle to retrieve async result (blocks on get())
std::promise	Sender/Producer	Write-once channel to set value or exception
std::async	Task Launcher	Combines thread creation + future generation

Basic Relationship:

```
// std::async (easiest):
std::future<int> fut = std::async([]{ return 42; });
int result = fut.get();  // Blocks until ready

// std::promise + std::future (manual control):
std::promise<int> prom;
std::future<int> fut = prom.get_future();

std::thread t([prom = std::move(prom)]() mutable {
    prom.set_value(42);  // Send value to future
});

int result = fut.get();  // Receive value
t.join();
```

Key Benefits:

- ✅ Value propagation across threads without explicit mutexes
- ✅ Automatic exception propagation
- ✅ RAII-based thread lifetime management (for async)
- ✅ Type-safe result delivery

Async, Promise, and Future - Asynchronous Task Management

Theory

📌 2. std::async - Launch Policies (1/2)

Policy Comparison:

Policy	Thread Creation	Execution Timing	Use Case
launch::async	✅ Guaranteed new thread	Immediate, parallel	True parallelism needed
launch::deferred	❌ No thread	Lazy (runs on get/wait)	Optional computation
Default (async|deferred)	⚠️ Implementation choice	Unpredictable	❌ Avoid - portability hazard

Critical: Default Policy is Dangerous

```
// ❌ AVOID: Default policy
auto fut = std::async(task);  // May run synchronously!

// ✅ EXPLICIT: Always specify
auto fut1 = std::async(std::launch::async, task);     // Guaranteed parallel
auto fut2 = std::async(std::launch::deferred, task);  // Guaranteed lazy
```

Lifetime Management:

Primitive	join/detach Required?	Lifetime Management
std::thread	✅ Must call join() or detach()	Manual
std::async	❌ Automatic via future destructor	RAII (but blocks!)

Async, Promise, and Future - Asynchronous Task Management

Theory

📌 2. std::async - Launch Policies (2/2)

Future Destructor Blocking:

```
{
    auto fut = std::async(std::launch::async, []{
        std::this_thread::sleep_for(std::chrono::seconds(5));
        return 42;
    });
    // ❌ fut destructor BLOCKS HERE for 5 seconds!
}
// Execution continues only after task completes
```

Async, Promise, and Future - Asynchronous Task Management

Theory

📌 3. Promise-Future Communication Channel

Single-Use Semantics:

Operation	Effect	Can Call Again?
promise.set_value()	Set result, unblock future	❌ Throws promise_already_satisfied
promise.set_exception()	Set exception for future	❌ Throws promise_already_satisfied
future.get()	Retrieve value, invalidate future	❌ Throws future_error: no_state
future.wait()	Block until ready (no value retrieval)	✅ Can call multiple times

Internal Synchronization:

```
// No manual mutex needed!
std::promise<int> prom;
std::future<int> fut = prom.get_future();

// Thread 1 (producer):
data = 42;
prom.set_value(100);  // Automatically synchronizes

// Thread 2 (consumer):
int val = fut.get();  // Sees data = 42 (happens-before relationship)
```

Happens-Before Guarantee:

Event	Synchronization
promise.set_value()	synchronizes-with → future.get()
All operations before set_value()	visible to operations after get()

Async, Promise, and Future - Asynchronous Task Management

Theory

📌 4. std::future vs std::shared_future

Single vs Multiple Consumers:

Feature	std::future	std::shared_future
Copyable	❌ Move-only	✅ Copyable
get() calls	Single-use (invalidates)	Multiple non-destructive reads
Use Case	Single consumer	Broadcast to multiple consumers
Conversion	`fut.share()`	Construct from future

Example: Broadcasting Results

```
std::promise<int> prom;
std::shared_future<int> shared_fut = prom.get_future().share();

// Multiple threads can all read:
auto reader = [shared_fut]() {
    int val = shared_fut.get();  // ✅ Non-destructive
    process(val);
};

std::thread t1(reader);
std::thread t2(reader);
std::thread t3(reader);

prom.set_value(42);  // All threads unblock

t1.join(); t2.join(); t3.join();
```

Async, Promise, and Future - Asynchronous Task Management

Theory

📌 5. std::packaged_task - Thread Pool Building Block

Purpose:

Separates task creation from execution, enabling thread pool implementations.

Workflow:

Step	Action	Component
1	Wrap callable	`packaged_task<RetType()> task(callable)`
2	Get future	`auto fut = task.get_future()`
3	Enqueue task	Add to thread pool queue
4	Worker executes	Worker calls `task()`
5	Retrieve result	Caller does `fut.get()`

Example:

```
// Create task:
std::packaged_task<int(int)> task([](int x) { return x * 2; });
std::future<int> fut = task.get_future();

// Execute later (possibly in another thread):
std::thread t(std::move(task), 21);

// Retrieve result:
int result = fut.get();  // 42
t.join();
```

Async, Promise, and Future - Asynchronous Task Management

Theory

📌 6. Timeouts and Waiting

Non-Blocking Wait Operations:

Operation	Return Type	Behavior
wait_for(duration)	future_status	Wait up to duration
wait_until(timepoint)	future_status	Wait until absolute time

future_status Values:

Status	Meaning
ready	Result available
timeout	Not ready yet (time expired)
deferred	Deferred task not executed yet

Example: Real-Time Deadline

```
auto fut = std::async(std::launch::async, sensor_processing);

if (fut.wait_for(std::chrono::milliseconds(100)) == std::future_status::ready) {
    auto result = fut.get();
    use_result(result);
} else {
    // Timeout: use fallback data
    use_previous_frame();
}
```

Async, Promise, and Future - Asynchronous Task Management

Theory

📌 7. Exception Propagation

Automatic Cross-Thread Exception Transfer:

```
// Producer throws:
auto fut = std::async(std::launch::async, [] {
    throw std::runtime_error("Sensor failure!");
    return 42;
});

// Exception stored (NOT thrown yet)
std::this_thread::sleep_for(std::chrono::seconds(1));

// Consumer retrieves:
try {
    int val = fut.get();  // ⚡ Exception rethrown HERE
} catch (const std::runtime_error& e) {
    handle_sensor_failure(e);
}
```

Manual Exception Setting:

```
std::promise<int> prom;
std::future<int> fut = prom.get_future();

try {
    int result = risky_operation();
    prom.set_value(result);
} catch (...) {
    prom.set_exception(std::current_exception());  // Forward exception
}
```

Async, Promise, and Future - Asynchronous Task Management

Theory

📌 8. Real-World: Autonomous Driving (1/2)

Sensor Processing Pipeline:

Component	Implementation	Benefit
Camera Capture	`std::async(process_frame)` returns `future<DetectionResult>`	Parallel frame processing
LiDAR Processing	Thread pool with `packaged_task`	Efficient worker utilization
Calibration Init	`promise` set by callback thread	Clean async initialization
Planning Coordination	`shared_future` for map updates	Broadcast to multiple modules
Real-Time Deadlines	`wait_for()` with 100ms timeout	Bounded latency guarantees

Example: Multi-Sensor Fusion

```
// Launch parallel sensor processing:
auto camera_fut = std::async(std::launch::async, process_camera);
auto lidar_fut = std::async(std::launch::async, process_lidar);
auto radar_fut = std::async(std::launch::async, process_radar);

// Collect results with timeout:
auto deadline = std::chrono::steady_clock::now() + std::chrono::milliseconds(50);

CameraData cam_data;
if (camera_fut.wait_until(deadline) == std::future_status::ready) {
    cam_data = camera_fut.get();
} else {
    cam_data = use_previous_camera_frame();
}

// Similar for lidar and radar...
fuse_sensor_data(cam_data, lidar_data, radar_data);
```

Async, Promise, and Future - Asynchronous Task Management

Theory

📌 8. Real-World: Autonomous Driving (2/2)

Why This Matters:

Metric	Raw Threads	std::async + futures
Code Complexity	High (manual sync)	Low (automatic sync)
Exception Handling	Manual error codes	Automatic propagation
Lifetime Management	Manual join/detach	RAII via future
Result Retrieval	Shared variables + mutex	Type-safe get()
Testability	Hard (threading exposed)	Easy (returns future)

Async, Promise, and Future - Asynchronous Task Management

Theory

📌 9. Common Pitfalls

Pitfall Table:

Mistake	Problem	Fix
Default launch policy	May run synchronously!	Always specify `launch::async` or `launch::deferred`
Multiple get() calls	Throws `future_error: no_state`	Use `shared_future` for multiple consumers
Ignoring future destructor	Unexpected blocking	Explicitly `get()` or `wait()` before scope exit
Promise not set	Broken promise exception or hang	Use RAII wrapper to set default value
Forgetting exceptions	Silent failures	Always `get()` to detect errors
Calling get() on packaged_task without executing	Hangs forever	Execute task before calling `get()`

Best Practices:

- ✅ Always specify explicit launch policy (`launch::async` or `launch::deferred`)
- ✅ Store futures from `std::async` to control lifetime
- ✅ Use `shared_future` for multiple consumers
- ✅ Always call `get()` to detect exceptions
- ✅ Use timeouts (`wait_for`) in real-time systems
- ✅ Prefer `std::async` for simple tasks, `packaged_task` for thread pools
- ❌ Never rely on default launch policy
- ❌ Never call `get()` twice on same future

Async, Promise, and Future - Asynchronous Task Management

Edge Cases

📌 1. Future Destructor Blocking Behavior

```
void example() {
    {
        std::future<int> fut = std::async(std::launch::async, []() {
            std::this_thread::sleep_for(std::chrono::seconds(5));
            return 42;
        });
        // ❌ fut destructor blocks here for 5 seconds!
    }
    std::cout << "Future destroyed\n";  // Prints after 5 seconds
}
```

When a future returned from std::async with launch::async policy is destroyed without calling get() or wait(), the destructor blocks until the task completes. This surprising behavior is unique to futures from async—manually created promise-future pairs do not block. The rationale is to prevent detached threads, but it can cause unexpected latency. To avoid blocking, explicitly call get() or store futures in containers to control lifetime.

Async, Promise, and Future - Asynchronous Task Management

Edge Cases

📌 2. Calling get() Multiple Times

```
std::promise<int> prom;
std::future<int> fut = prom.get_future();

prom.set_value(42);

int val1 = fut.get();  // ✅ val1 = 42
int val2 = fut.get();  // ❌ Throws std::future_error: no_state
```

std::future is single-use: get() transfers the value out and invalidates the future. A second get() throws future_error with error code future_errc::no_state because the shared state has been consumed. If multiple threads need the result, use std::shared_future which allows multiple get() calls through copying. Convert a future to shared_future via fut.share() or the shared_future constructor.

Async, Promise, and Future - Asynchronous Task Management

Edge Cases

📌 3. Promise Destroyed Without Setting Value

```
std::future<int> create_broken_future() {
    std::promise<int> prom;
    std::future<int> fut = prom.get_future();
    return fut;  // ❌ prom destroyed without set_value!
}

void caller() {
    auto fut = create_broken_future();
    try {
        int val = fut.get();  // Throws std::future_error: broken_promise
    } catch (const std::future_error& e) {
        std::cout << "Broken promise: " << e.what() << "\n";
    }
}
```

If a promise is destroyed without calling set_value() or set_exception(), the future receives a broken_promise exception. This prevents indefinite blocking when the promise is abandoned. In production code, ensure promises are moved to the worker thread or stored in a scope that guarantees set_value() is called. RAII wrappers can automate this: destructor sets a default value or exception if not already set.

Async, Promise, and Future - Asynchronous Task Management

Edge Cases

📌 4. Launch Policy Ambiguity with Default Policy

```
auto fut = std::async([]() { return expensive_computation(); });
// ❌ Might run synchronously on get()!

// fut.get() may execute the task in the current thread
auto result = fut.get();  // Could be synchronous!
```

The default launch policy (std::launch::async | std::launch::deferred) allows the implementation to choose. Most standard libraries use deferred, meaning the task runs lazily on first get() or wait() in the calling thread. This defeats the purpose of async execution. Always explicitly specify std::launch::async for true asynchronous execution, or std::launch::deferred for guaranteed lazy evaluation. Relying on default policy is a portability hazard.

Async, Promise, and Future - Asynchronous Task Management

Edge Cases

📌 5. Exception Propagation Timing

```
std::future<int> fut = std::async(std::launch::async, []() {
    throw std::runtime_error("Task failed");
    return 42;
});

// Exception stored in shared state, not thrown immediately

std::this_thread::sleep_for(std::chrono::seconds(1));
// Task already threw exception, but no sign yet

try {
    int val = fut.get();  // ⚡ Exception rethrown here
} catch (const std::runtime_error& e) {
    std::cout << "Caught: " << e.what() << "\n";
}
```

Exceptions thrown in async tasks or set via set_exception() are stored in the shared state and rethrown when get() is called. The exception does not propagate immediately when thrown—it waits until the future is consumed. This deferred exception delivery means errors are not discovered until results are requested, potentially hiding failures in fire-and-forget scenarios. Always call get() or wait() to ensure task completion and error detection.

Async, Promise, and Future - Asynchronous Task Management

Code Examples

📌 1. Basic std::async with Launch Policy

This demonstrates std::async with explicit launch::async policy ensuring true asynchronous execution. The main thread continues work while the computation runs in parallel. The get() call blocks until the result is available, but work can proceed beforehand. For autonomous vehicle perception, this pattern allows launching multiple sensor processing tasks and collecting results later.

```
#include <future>
#include <iostream>
#include <chrono>

int expensive_computation() {
    std::this_thread::sleep_for(std::chrono::seconds(2));
    return 42;
}

int main() {
    // ✅ Explicitly async: runs in new thread
    std::future<int> fut = std::async(std::launch::async, expensive_computation);

    std::cout << "Computation started, doing other work...\n";
    std::this_thread::sleep_for(std::chrono::seconds(1));

    std::cout << "Getting result...\n";
    int result = fut.get();  // Blocks until ready
    std::cout << "Result: " << result << "\n";

    return 0;
}
```

Async, Promise, and Future - Asynchronous Task Management

Code Examples

📌 2. std::promise and std::future for Thread Communication

This pattern manually creates a promise-future pair for explicit thread communication. The promise is moved into the worker thread where set_value() signals completion. This is more flexible than async for scenarios requiring custom thread management, such as sensor callback threads in autonomous vehicle middleware that must run on specific cores for real-time guarantees.

```
#include <future>
#include <thread>
#include <iostream>

void worker_thread(std::promise<int> prom) {
    std::this_thread::sleep_for(std::chrono::seconds(1));

    // Simulate computation
    int result = 123;

    // Send result via promise
    prom.set_value(result);  // ✅ Value delivered to future
}

int main() {
    std::promise<int> prom;
    std::future<int> fut = prom.get_future();

    // Launch worker with moved promise (promises are move-only)
    std::thread worker(worker_thread, std::move(prom));

    std::cout << "Waiting for result...\n";
    int value = fut.get();  // Blocks until set_value() called
    std::cout << "Received: " << value << "\n";

    worker.join();
    return 0;
}
```

Async, Promise, and Future - Asynchronous Task Management

Code Examples

📌 3. Exception Propagation Through Future

Exceptions thrown in async tasks are captured and stored in the shared state. When get() is called, the exception is rethrown in the calling thread. This provides clean exception propagation without manual error code handling. In autonomous driving perception pipelines, sensor processing failures (invalid calibration, missing data) propagate to the planning layer as exceptions rather than special error values.

```
#include <future>
#include <stdexcept>
#include <iostream>

int risky_operation() {
    throw std::runtime_error("Operation failed!");
    return 0;  // Never reached
}

int main() {
    std::future<int> fut = std::async(std::launch::async, risky_operation);

    std::cout << "Task launched\n";

    try {
        int result = fut.get();  // ⚡ Exception rethrown here
    } catch (const std::runtime_error& e) {
        std::cout << "Caught exception: " << e.what() << "\n";
    }

    return 0;
}
```

Async, Promise, and Future - Asynchronous Task Management

Code Examples

📌 4. std::shared_future for Multiple Consumers

std::shared_future allows multiple threads to retrieve the same value. Unlike std::future which is invalidated by get(), shared_future can be copied and get() called multiple times. This is useful for broadcasting results to multiple consumers, such as updated HD map data distributed to perception, planning, and visualization modules in autonomous vehicles.

```
#include <future>
#include <thread>
#include <iostream>
#include <vector>

int main() {
    std::promise<int> prom;
    std::shared_future<int> shared_fut = prom.get_future().share();  // ✅ Convert to shared

    // Multiple threads can call get() on shared_future
    auto reader = [shared_fut]() {
        int val = shared_fut.get();  // ✅ Non-destructive read
        std::cout << "Thread " << std::this_thread::get_id()
                  << " read: " << val << "\n";
    };

    std::vector<std::thread> readers;
    for (int i = 0; i < 3; ++i) {
        readers.emplace_back(reader);
    }

    std::this_thread::sleep_for(std::chrono::milliseconds(100));
    prom.set_value(42);  // All waiting threads unblock

    for (auto& t : readers) {
        t.join();
    }

    return 0;
}
```

Async, Promise, and Future - Asynchronous Task Management

Code Examples

📌 5. Deferred Execution with std::launch::deferred

With launch::deferred, the task does not execute until get() or wait() is called, and it runs synchronously in the calling thread. This is useful for lazy evaluation where you want to defer expensive computations until needed. In autonomous driving, deferred tasks can represent optional high-resolution map queries that only execute if the planning layer requests them.

```
#include <future>
#include <iostream>

int lazy_computation(int x) {
    std::cout << "Computation executing now\n";
    return x * 2;
}

int main() {
    // ✅ Deferred: computation only runs on get()
    auto fut = std::async(std::launch::deferred, lazy_computation, 21);

    std::cout << "Future created, but computation not started\n";
    std::this_thread::sleep_for(std::chrono::seconds(1));
    std::cout << "Still not started...\n";

    std::cout << "Calling get()...\n";
    int result = fut.get();  // Computation executes NOW in this thread
    std::cout << "Result: " << result << "\n";

    return 0;
}
```

Async, Promise, and Future - Asynchronous Task Management

Code Examples

📌 6. Waiting with Timeout

wait_for() and wait_until() allow non-blocking checks with timeouts. If the future is not ready, the calling thread is not blocked indefinitely. This is critical for real-time autonomous driving systems where perception cycles have hard deadlines (e.g., 100ms). If a lidar processing task exceeds its budget, the system can timeout and use fallback data rather than blocking the entire pipeline.

```
#include <future>
#include <iostream>
#include <chrono>

int slow_task() {
    std::this_thread::sleep_for(std::chrono::seconds(5));
    return 42;
}

int main() {
    auto fut = std::async(std::launch::async, slow_task);

    std::cout << "Waiting for result with timeout...\n";

    if (fut.wait_for(std::chrono::seconds(2)) == std::future_status::ready) {
        std::cout << "Result: " << fut.get() << "\n";
    } else {
        std::cout << "Timeout! Task still running.\n";
        // ⚠️ Future still valid, can wait again or get() later
        fut.wait();  // Wait until completion
        std::cout << "Result after wait: " << fut.get() << "\n";
    }

    return 0;
}
```

Async, Promise, and Future - Asynchronous Task Management

Code Examples

📌 7. std::packaged_task for Deferred Execution

std::packaged_task wraps a callable and associates it with a future. Unlike async which launches immediately, packaged_task allows manual control over when and where execution happens. This is useful for thread pools: tasks are created, queued, and workers execute them when available. In autonomous vehicle thread pools, packaged_tasks represent individual camera frame processing jobs with futures for collecting results.

```
#include <future>
#include <iostream>

int compute(int x, int y) {
    return x + y;
}

int main() {
    // ✅ packaged_task wraps callable and provides future
    std::packaged_task<int(int, int)> task(compute);
    std::future<int> fut = task.get_future();

    // Task can be executed later, possibly in another thread
    std::thread t(std::move(task), 10, 20);  // Execute in thread

    std::cout << "Waiting for result...\n";
    int result = fut.get();
    std::cout << "Result: " << result << "\n";

    t.join();
    return 0;
}
```

Async, Promise, and Future - Asynchronous Task Management

Code Examples

📌 8. Setting Exception via Promise

When a promise-based worker encounters an error, call set_exception() with std::current_exception() to forward the exception to the future. This maintains exception propagation across thread boundaries without losing error information. For autonomous vehicle sensor drivers, calibration failures or hardware errors can be propagated to the perception layer as exceptions via promises.

```
#include <future>
#include <thread>
#include <iostream>
#include <stdexcept>

void worker(std::promise<int> prom, bool succeed) {
    try {
        if (!succeed) {
            throw std::runtime_error("Task failed");
        }
        prom.set_value(42);  // Success case
    } catch (...) {
        // ✅ Capture and forward exception to future
        prom.set_exception(std::current_exception());
    }
}

int main() {
    std::promise<int> prom;
    std::future<int> fut = prom.get_future();

    std::thread t(worker, std::move(prom), false);  // Fail mode

    try {
        int val = fut.get();  // ⚡ Exception rethrown
        std::cout << "Value: " << val << "\n";
    } catch (const std::runtime_error& e) {
        std::cout << "Caught: " << e.what() << "\n";
    }

    t.join();
    return 0;
}
```

What You Learned

Key Takeaways

- ✓ Overview
- ✓ Launch Policies
- ✓ Promise-Future Communication Channel
- ✓ std::future vs std::shared_future

→ Swipe to learn →